Local AI Orchestrator

Onboarding — how to use it

Practical, command-by-command guide. Pair with the System Guide for the why and how.

Generated: 2026-05-17

Table of contents

1. Quick reference

Every common operation, one row. Run all commands from the project root: `cd /Users/macbookpro/Projects/local-ai-orchestrator`.

Goal	Command
See system health (Docker, Ollama, ports)	bash scripts/check_system.sh
Start Docker services	bash scripts/start_services.sh
Stop Docker services (keep data)	bash scripts/stop_services.sh
Start Ollama (persistent service)	brew services start ollama
Stop Ollama	brew services stop ollama
Start the control panel	bash scripts/start_control_panel.sh
Stop the control panel	kill $(lsof -nP -iTCP:8501 -sTCP:LISTEN -t)
Start LiteLLM (router)	bash scripts/start_litellm.sh
Start Temporal worker	bash scripts/start_temporal_worker.sh
Open the panel in browser	http://127.0.0.1:8501
Open WebUI for chat with local models	http://localhost:3000
Open Temporal UI	http://localhost:8233
Create a work packet from CLI	bash scripts/create_work_packet.sh "Title" "Description"
Submit answers to clarification	bash scripts/answer_questions.sh <id> path/to/answers.md
Allow local models in the day	bash scripts/day_unlock.sh "reason"
Lock the day-unlock again	bash scripts/day_lock.sh
Load/unload/test a model in browser	Panel -> Models tab
Edit day/night times in browser	Panel -> Schedule tab
Verify nothing runs automatically	launchctl list | grep localai (empty = off)
List currently-pulled models	ollama list
See what is in VRAM right now	ollama ps
Pull all configured models (~41 GB)	bash scripts/pull_models.sh
Force-unload a model right now	ollama stop <tag>
Run the full test suite	bash scripts/run_tests.sh
Regenerate these PDFs	.venv/bin/python scripts/build_pdfs.py

2. Daily start sequence

Every time you want to use the system, run these four steps in order. None of them load models or do heavy work; they just bring services online.

```
cd /Users/macbookpro/Projects/local-ai-orchestrator

# 1. Make sure Ollama is running (persistent service)
brew services start ollama
curl http://localhost:11434/api/tags     # should return JSON

# 2. Start Docker services (Postgres, Temporal, Open WebUI)
bash scripts/start_services.sh

# 3. Confirm everything is up
bash scripts/check_system.sh

# 4. Start the control panel
bash scripts/start_control_panel.sh
# then browse to http://127.0.0.1:8501
```

All four are safe to re-run. start_services.sh skips already-running containers; brew services start ollama is a no-op if it's already loaded; start_control_panel.sh will fail if port 8501 is held by a previous Streamlit (kill it first with kill $(lsof -nP -iTCP:8501 -sTCP:LISTEN -t)).

3. Daily shutdown sequence

End of day, in order. Each step is independent.

```
# 1. Lock the day-unlock if it was on
bash scripts/day_lock.sh

# 2. Stop the Docker services (keeps volumes, your packets persist)
bash scripts/stop_services.sh

# 3. Stop the control panel
kill $(lsof -nP -iTCP:8501 -sTCP:LISTEN -t) 2>/dev/null

# 4. Optional: stop Ollama (lets the M-series unified memory page out)
brew services stop ollama

# 5. Optional: stop LiteLLM and the Temporal worker if running
kill $(lsof -nP -iTCP:4000 -sTCP:LISTEN -t) 2>/dev/null   # LiteLLM
pkill -f temporal_app.worker                                # worker
```

stop_services.sh stops Docker containers ONLY. Streamlit, Ollama, LiteLLM, and the Temporal worker run as host processes and are NOT affected. That is why you can still see the Streamlit panel at :8501 after stopping services — the panel just shows database errors because Postgres is down.

4. A complete worked example (vague to ready)

End-to-end use of the system today, from a vague request to a ready packet. Execution is still scaffolded; this stops before the night step.

Step 1: create a vague packet (expect rejection)

```
bash scripts/create_work_packet.sh "Tomorrow prep" \
   "Prepare me for tomorrow from my notes."
```

Expected output:

```
{
  "work_packet_id": "abc123...",
  "status": "UNDERDEFINED",
  "readiness_score": 0.18,
  "questions": [
    {"category": "objective",     "question": "What exact outcome..."},
    {"category": "outputs",       "question": "What output files..."},
    {"category": "sources",       "question": "Which folders are..."},
    {"category": "privacy/cloud", "question": "Is cloud allowed..."},
    ...
  ]
}
```

The score is below 0.65 and status is UNDERDEFINED. **This is correct.** The system refuses to act until you answer the gating questions.

Step 2: look at the sample answer format

```
cat examples/sample_answers.md
```

Step 3: edit your own answers or reuse the sample

Write a markdown file with answers. Specific paths under ~/LocalAI/inbox/, explicit cloud policy, clear quality bar, named audience.

Step 4: submit the answers

```
bash scripts/answer_questions.sh abc123... examples/sample_answers.md
```

Expected output:

```
{
  "work_packet_id": "abc123...",
  "status": "READY_FOR_OVERNIGHT",
  "readiness_score": 0.90,
  "database": "updated",
  "remaining_questions": []
}
```

Status flipped, no remaining questions. The packet is eligible for overnight execution.

Today, the actual overnight execution is scaffolded. Submitting overnight runs creates an execution_runs row and queues a Temporal workflow that returns a placeholder. Wiring the real pipeline is the next major chunk of feature work.

5. Using the Models tab

From http://127.0.0.1:8501, click **Models**. You'll see three sections:

Currently loaded (in VRAM)

What Ollama has resident right now. Each row shows the tag, VRAM size, and an expires_at timestamp (when Ollama's keep-alive timer will auto-unload). An **Unload** button frees VRAM immediately.

Pulled models (on disk)

What's available to load. Each row shows the tag, the model group it maps to, size, and modification time. **Load** pulls weights into VRAM with a 30-minute keep-alive. Buttons are disabled in strict DAY_MODE; if you need to load during the day, run bash scripts/day_unlock.sh "reason" first.

Quick test prompt

Single-shot verification. Pick a loaded model, type a prompt, hit Send. The response renders in a code block. Not a chat — no history. For conversations, use Open WebUI at http://localhost:3000.

> First inference after a fresh Load is always the slowest (KV cache cold). Subsequent prompts are 5-10x faster. With two 30B models loaded at once (61.6 GB VRAM), expect noticeable slowdown from memory pressure on a 96 GB M2 Max.

6. Using the Schedule tab

From the panel, click **Schedule**. Two sections:

Status metrics

Three independent checks of the launchd job: is the project plist present, is the plist installed in ~/Library/LaunchAgents, is it loaded in launchctl. All three off = auto-execution is OFF.

Edit times

Three editable fields: day_mode_start, night_mode_start, night_mode_end. Format 24-hour HH:MM. Click Save to rewrite config/assistant.yaml atomically. Validation runs first; bad values are rejected with a clear error.

> Editing the times does NOT auto-flip LOCALAI_MODE. That stays an explicit env-var gesture. The times drive (a) the launchd plist when you regenerate it, and (b) the visual hint in the header showing whether you're in DAY or NIGHT window right now.

If the plist is already installed

The panel shows a yellow warning with the three commands needed to regenerate and reload the plist so the new times take effect. The panel never touches launchd directly — arming and disarming are deliberate terminal commands.

7. Day-unlock switch

DAY_MODE blocks every local-* model group. To deliberately allow local models during the day, the day-unlock switch flips a single, visible boolean.

Turn it on

```
bash scripts/day_unlock.sh "checking last night's output"
```

Creates ~/LocalAI/state/day_unlock.flag with a timestamp, hostname, and reason. The Streamlit header flips Day unlock from off to ACTIVE and a red banner appears with the flag contents.

Turn it off

```
bash scripts/day_lock.sh
```

Deletes the flag. Warns if LOCALAI_DAY_UNLOCK is also set in the shell so you know to unset it.

One-shot env-var variant

```
LOCALAI_DAY_UNLOCK=true .venv/bin/python -m assistant_core.cli sample-readiness
```

Per-shell, ephemeral. Useful for one-off commands without touching the flag file.

> Day unlock does NOT open the cloud path. cloud-claude-opus still raises a SafetyError regardless. Full design rationale: docs/day_unlock.md.

8. Recipe: morning brief

The flagship use case. Goal: tomorrow morning, you read one markdown file and know what matters.

1. Drop your inputs

```
# Drop yesterday's working materials into the inbox
cp ~/Downloads/2026-W20-recap.md       ~/LocalAI/inbox/notes/
cp ~/Downloads/2026-05-15-1on1.md      ~/LocalAI/inbox/transcripts/
cp ~/Downloads/q3-staffing-draft.md    ~/LocalAI/inbox/docs/
```

2. Create the packet

```
bash scripts/create_work_packet.sh \
   "Morning prep 2026-05-18" \
   "Prepare a private morning brief from this week's notes, transcripts, and docs. S
urface priorities, decisions needed, risks, and meeting prep."
```

3. Answer the questions

Edit a copy of examples/sample_answers.md; see section 13 for the exact text I'd suggest. Then:

```
bash scripts/answer_questions.sh <packet_id> ~/answers/morning-2026-05-18.md
```

4. Verify ready

Open the panel -> Dashboard. The packet should show READY_FOR_OVERNIGHT or READY_HIGH_CONFIDENCE.

5. Wait for night

If launchd auto-execution is armed: nothing to do. If not, the brief generates when you run LOCALAI_MODE=NIGHT_MODE bash scripts/run_ready_overnight.sh.

> Once the execution graph is wired (next chunk of feature work), the output lands in ~/LocalAI/output//01_MORNING_BRIEF.md and a copy in ~/Obsidian/LocalAI-ChiefOfStaff/01_Daily_Briefs/.

9. Recipe: meeting prep

Goal: a single page for each upcoming meeting with the right context, agenda, stakeholder notes, and "things to push for / things to listen for."

Setup

```
# Stage the inputs
cp ~/Downloads/2026-05-16-board-deck-draft.md  ~/LocalAI/inbox/docs/
cp ~/Downloads/prior-board-notes.md             ~/LocalAI/inbox/notes/
cp ~/Downloads/q2-financials-narrative.md      ~/LocalAI/inbox/docs/
```

Create packet

```
bash scripts/create_work_packet.sh \
   "Board prep 2026-05-20" \
   "Prepare for the board meeting. Cover the Q2 narrative, any open decisions, and l
ikely tough questions. High-stakes."
```

Note: the words "board" and "high-stakes" trigger high_stakes=True on the packet. The threshold becomes 0.90 (not 0.85) before execution.

Answers worth giving

```
Outputs: meeting prep markdown file, decisions needed list,
  one-page narrative summary.
Sources: only ~/LocalAI/inbox/docs and ~/LocalAI/inbox/notes.
  Treat Q2 financial narrative as authoritative.
Cloud: do not use cloud fallback.
Audience: me, then I share with the board.
Assumptions: do not infer financial figures; surface ambiguity
  rather than guess.
Quality: optimize for factuality and defensibility. No hedging.
Stop conditions: stop if any figure is unsourced, or if a decision
  conflicts with a prior board minute.
Success: I can walk in and answer every question without notes.
```

10. Recipe: decision capture from transcripts

Goal: read meeting transcripts, extract the decisions, who owns each follow-up, and write them to the decisions vault.

Setup

```
# Drop the transcripts (use OpenAI Whisper or similar to transcribe)
cp ~/Downloads/2026-05-week-transcripts/*.md \
    ~/LocalAI/inbox/transcripts/
```

Create packet

```
bash scripts/create_work_packet.sh \
   "Decision capture 2026-W20" \
   "Read this week's call transcripts. Extract every decision made, every action ite
m with owner, and any items where ownership was left ambiguous. Write to my decision
s vault."
```

Answers worth giving

```
Outputs: decisions markdown, action items grouped by owner,
  separate file flagging ambiguous ownership.
Sources: only ~/LocalAI/inbox/transcripts.
Cloud: do not use cloud fallback.
Audience: me, then propagated to Obsidian 05_Decisions/.
Assumptions: do NOT infer ownership when not stated. Flag instead.
Quality: optimize for grounding. Every decision must cite a transcript
  line range.
Stop conditions: stop if a decision references a person not present
  in the call.
Success: every decision is traceable to its transcript moment.
```

11. Recipe: risk and blocker surfacing

Goal: read recent notes/logs, flag anything that looks like a risk or blocker for review tomorrow morning.

Setup

```
# A week's worth of notes
cp ~/Downloads/2026-W20-*.md ~/LocalAI/inbox/notes/
```

Create packet

```
bash scripts/create_work_packet.sh \
   "Risk scan 2026-W20" \
   "Read this week's notes and flag risks, blockers, and anything that looks like it
 could break next week. Sort by severity."
```

Answers worth giving

```
Outputs: 05_RISKS_AND_BLOCKERS.md with severity + recommended action
  for each item.
Sources: only ~/LocalAI/inbox/notes from this week.
Cloud: do not use cloud fallback.
Audience: me only.
Assumptions: rate severity using a low/medium/high scale. Label
  assumptions about external context (e.g., vendor timelines).
Quality: optimize for completeness over polish. Better to over-flag.
Stop conditions: never stop &mdash; flag everything, even uncertain.
Success: nothing important slipped through this week.
```

12. Recipe: code task using qwen3-coder

Goal: feed a code task to local-coder (qwen3-coder:30b) for structured review, refactor proposal, or test generation.

Setup

```
# Copy the file or directory into the inbox
cp ~/Projects/some-repo/src/payments.py  ~/LocalAI/inbox/docs/
cp ~/Projects/some-repo/tests/payments_test.py  ~/LocalAI/inbox/docs/
```

Create packet

```
bash scripts/create_work_packet.sh \
   "Payments refactor review" \
   "Review payments.py for clarity issues, suggest a refactor that improves testabil
ity, and propose three additional test cases that the current tests miss."
```

Answers worth giving

```
Outputs: review markdown with three sections: clarity issues,
  refactor proposal (with code), additional tests (with code).
Sources: ~/LocalAI/inbox/docs/payments.py and payments_test.py.
Cloud: do not use cloud fallback.
Audience: me, then I share with a teammate.
Assumptions: stay within the existing dependency set. Do not propose
  introducing new libraries.
Quality: optimize for actionability. Each suggestion must be applyable
  in under 10 minutes.
Stop conditions: stop if the file references a config file not provided.
Success: I can apply the suggestions today and ship.
```

Today, the orchestrator routes code-classified tasks to local-coder automatically. The classification is part of the (scaffolded) execution graph. Until that's wired, you can manually load qwen3-coder:30b in the Models tab and quick-test prompt the review.

13. Sample answer files for every recipe

Copy these as starting points; adapt them to your specific work. The parser is forgiving about wording but cares about explicit cloud policy, explicit paths under ~/LocalAI/inbox/, and recognizable key labels (Outputs, Sources, Cloud, Audience, Quality, Assumptions, Stop conditions, Success).

Morning brief template

```
Outputs: 01_MORNING_BRIEF.md, 02_TODAY_PRIORITIES.md,
  04_DECISIONS_NEEDED.md, 05_RISKS_AND_BLOCKERS.md,
  07_MEETING_PREP.md.
Sources: only ~/LocalAI/inbox/notes and ~/LocalAI/inbox/transcripts
  and ~/LocalAI/inbox/docs. Skip anything older than one week.
Cloud: do not use cloud fallback.
Audience: me only.
Assumptions: infer low-risk priorities but label every assumption.
Quality: factuality > completeness > polish.
Stop conditions: stop for missing sources or unclear ownership.
Success: I read it in five minutes and know what to do first.
```

High-stakes meeting prep template

```
Outputs: 07_MEETING_PREP.md (one section per meeting), plus a
  separate one-page narrative summary for sharing.
Sources: only ~/LocalAI/inbox/docs and ~/LocalAI/inbox/notes.
Cloud: do not use cloud fallback.
Audience: me, then I share with named participants.
Assumptions: do not infer financial or commitment figures.
Quality: defensibility > brevity.
Stop conditions: stop if any figure is unsourced.
Success: I can answer any question without notes.
```

Decision capture template

```
Outputs: decisions.md, action_items_by_owner.md,
  ambiguous_ownership.md.
Sources: only ~/LocalAI/inbox/transcripts.
Cloud: do not use cloud fallback.
Audience: me, propagated to Obsidian 05_Decisions/.
Assumptions: do NOT infer ownership. Flag instead.
Quality: every decision cites a transcript line range.
Stop conditions: stop if a decision references a person not in the call.
Success: every decision is traceable to its transcript moment.
```

Risk scan template

```
Outputs: 05_RISKS_AND_BLOCKERS.md sorted by severity, with
  recommended action per item.
Sources: only ~/LocalAI/inbox/notes from this week.
Cloud: do not use cloud fallback.
Audience: me only.
Assumptions: rate severity low/medium/high. Label assumptions.
Quality: completeness > polish. Over-flag.
```

```
Stop conditions: never stop; flag everything, even uncertain.
Success: nothing important slipped through.
```

Code task template

```
Outputs: review.md with three sections: clarity, refactor proposal,
  additional test cases.
Sources: ~/LocalAI/inbox/docs/<file>.py and any provided tests.
Cloud: do not use cloud fallback.
Audience: me, then a named teammate.
Assumptions: stay within existing dependency set.
Quality: actionability > thoroughness. Each suggestion applyable
  in under ten minutes.
Stop conditions: stop if the file references missing context.
Success: I can apply today.
```

14. Do / Don't

Do

- Keep input files under ~/LocalAI/inbox/. That's the only place this system reads from.
- Be specific in clarification answers. Explicit paths, single-line key:value lines parse better than prose.
- Mark packets as high_stakes (board, CEO, legal, investor) when they need the 0.90 readiness bar.
- Use day_unlock deliberately, lock again when done. The red banner is your reminder.
- Run bash scripts/run_tests.sh before committing changes.
- Read AGENTS.md if you're going to modify this repo.
- Pull models on stable Wi-Fi; qwen3:30b-a3b is ~18 GB.
- Unload models when you're done; 30 GB of idle VRAM is wasted budget.
- Keep your Obsidian vault open while reviewing memory candidates.

Don't

- Don't put ANTHROPIC_API_KEY in .env unless you actually want cloud fallback. Default policy refuses it; setting the key removes one of six gates.
- Don't point the system at ~/Documents, ~/Desktop, or anywhere outside ~/LocalAI/inbox/.
- Don't delete the temporal_data Docker volume thinking it holds your work. Your packets live in postgres_data.
- Don't edit files in ~/LocalAI/output/ and expect them to come back next run. Save important outputs into your Obsidian vault.
- Don't leave day_unlock on overnight if you don't need to. It persists across reboots by design.
- Don't bypass assert_model_allowed() by hitting Ollama/LiteLLM directly with raw tags. That defeats the safety policy.
- Don't start the Temporal worker before docker services are healthy.
- Don't load two 30B models simultaneously and expect snappy inference.
- Don't assume stop_services.sh stops everything — it stops Docker only.

15. Memory budgeting (what to load when)

Your M2 Max has 77.8 GB Metal VRAM. Comfortable budgets:

Scenario	Loaded models	VRAM	Why
Spot-check quality	local-reasoner only	~5 GB	Fast first inference. Good for one-off checks.
Drafting work	local-main only	~17 GB	30B MoE for general drafting. Comfortable margin.
Code review	local-coder only	~17 GB	Swap to coder when working on code tasks.
Overnight pipeline	local-main + local-reasoner	~22 GB	Primary model + evaluator. Default night profile.
Quality-gated retry	local-main, local-reasoner, local-secondary 70B loaded	~60 GB	70B loads when local-main output fails the evaluator.
Trying everything at once	all four loaded	~60+ GB	Don't. You'll feel it.

Keep-alive defaults: Load button = 30 minutes; Quick test = 5 minutes. Ollama auto-unloads idle models after the timer expires, so even if you forget to unload, VRAM frees up on its own within half an hour.

16. Troubleshooting

Symptom	Fix
Safari can't connect to 127.0.0.1:8501	Streamlit isn't running. bash scripts/start_control_panel.sh. If port is held: kill $(lsof -nP -
Error: could not connect to ollama server	Ollama isn't running. brew services start ollama. Verify with curl http://localhost:11434/ap
Docker shows Temporal Restarting	docker logs --tail=20 localai-temporal. If dynamic-config error, force-recreate: docker cor
Postgres unavailable	Docker isn't running or postgres container is down. bash scripts/start_services.sh
Day unlock banner won't go away	Refresh the browser. Check echo $LOCALAI_DAY_UNLOCK in the shell that started St
bash scripts/install_core.sh fails on Python 3.14	Use Python 3.13: brew install python@3.13 then LOCALAI_PYTHON_BIN=/opt/homebr
Model pull says "file does not exist"	Tag name wrong. Try alternatives listed at the end of scripts/pull_models.sh, or qwen2.5
Panel Unload doesn't unload	Refresh page. If still loaded, run ollama stop <tag> from terminal as a workaround.
Tests fail with state-dir errors	tests/conftest.py should isolate. If you've deleted it, restore it.
Quick test prompt takes 30+ seconds	First inference after a load is cold. Subsequent calls 5-10x faster. Also: are two 30B mod
Open WebUI won't open at :3000	Container is part of the Docker stack. Run bash scripts/start_services.sh.
Brew says ollama started but curl fails	Race condition. Wait 3-5 seconds and retry. If still failing after 10s: brew services list | gr
I see two ollama serve processes	Probably one from brew + one from your earlier foreground run. brew services stop ollar

17. Recovery scenarios

Streamlit crashed mid-session

```
# Diagnose: nothing on the port? probably dead.
lsof -nP -iTCP:8501 -sTCP:LISTEN

# Restart
bash scripts/start_control_panel.sh

# Your work packets are in Postgres &mdash; nothing lost.
```

Ollama daemon died, models you had loaded are gone

```
brew services restart ollama
# Re-load the model you were testing
ollama list                          # confirm tag still on disk
# In the panel, click Load again, OR:
ollama run <tag> 'hello'              # forces load + responds
```

Temporal container is restarting

```
docker logs --tail=30 localai-temporal
# If 'dynamic config' error, the path in compose was wrong; should be
# config/dynamicconfig/docker.yaml (fixed in commit a2d6485).
docker compose up -d --force-recreate temporal
```

I think launchd is running my packets but I didn't arm it

```
launchctl list | grep com.localai.orchestrator.nightly
# Empty output = not loaded. You're not armed.
# If something is in the output:
launchctl unload \
  ~/Library/LaunchAgents/com.localai.orchestrator.nightly.plist
```

I accidentally pulled a 70B model on slow Wi-Fi

```
# Cancel the pull
pkill -f 'ollama pull llama3.3'

# Optional: remove the partial layers from disk
ollama rm llama3.3:70b
```

I want to start completely fresh

```
# Stop everything
bash scripts/stop_services.sh
brew services stop ollama
kill $(lsof -nP -iTCP:8501 -sTCP:LISTEN -t) 2>/dev/null

# NUCLEAR: also wipe all volumes (you lose your work packets!)
docker compose down -v
```

```
# Then bring it all back up from scratch
bash scripts/start_services.sh
brew services start ollama
```

18. Weekly maintenance checklist

Five-minute weekly upkeep to keep the system healthy:

- **Empty the inbox**: archive consumed inputs from ~/LocalAI/inbox/ to ~/LocalAI/archive/ so next week's pulls are clean.
- **Review memory candidates**: open ~/Obsidian/LocalAI-ChiefOfStaff/00_Inbox/Memory_Review/ and approve or trash each candidate.
- **Prune old output directories**: ~/LocalAI/output/YYYY-MM-DD/ older than four weeks can be deleted. Anything important should already be in your vault.
- **Check disk usage**: du -sh ~/.ollama/models — if you have unused tags, ollama rm <tag> them.
- **Test suite**: bash scripts/run_tests.sh after any code change, even small ones.
- **Update if needed**: brew upgrade ollama — the keep-alive payload behavior changed across versions; we send '0s' string to be compatible.
- **Lock day-unlock**: bash scripts/day_lock.sh as a hygiene reset, even if you don't think it's on.
- **Backup your vault**: rsync, Time Machine, whatever you use. The orchestrator does not back up the vault for you.

19. Safety rules at a glance

All enforced in code, not just documented.

Rule	Where enforced
No cloud calls without 6 policy gates passing	assert_cloud_allowed
No local model calls in DAY_MODE by default	assert_heavy_execution_allowed + LOCAL_MODEL_GROUPS
Day unlock is the only relax mechanism, and it's visible	day_unlock_active + Streamlit banner
No writes outside allowed roots	assert_original_file_write_allowed
No emails / external API writes in v1	assert_no_external_write (always denies)
Cloud Claude blocked at model selection too	assert_model_allowed('cloud-claude-opus', ...) raises
Services bind to 127.0.0.1 only	docker-compose.yml port mappings
Original inbox files never modified	Output paths constrained to ~/LocalAI/output/
Nightly auto-execution OFF by default	launchd plist generated but not installed; arming is a 3-step manual g
Unknown model tags fall back to local-main	ollama_admin.resolve_group_for_tag (fail-closed)

20. Tips and tricks

- **Long pulls survive a closed terminal.** Wrap any ollama pull in nohup ... > ~/LocalAI/logs/pull-<tag>.log 2>&1 &. Close the window, come back later, check ollama list.
- **Parallel pulls share bandwidth.** Three pulls in parallel = each gets a third the speed. Total wall-clock the same. Serializing (one at a time) gets you the smallest model in front of you fastest.
- **Keep your laptop awake.** Use Amphetamine or nohup caffeinate -i & for unattended overnight pulls. Closed lids suspend network on Wi-Fi-only Macs.
- **The day-unlock flag is just a file.** If day_lock.sh can't reach you (you're SSH'd in, weird shell), rm ~/LocalAI/state/day_unlock.flag does the same thing.
- **Read the audit log first when debugging**: 09_AUDIT_LOG.md shows which models ran, what the evaluator said, and where each output came from.
- **Your Obsidian vault is the real long-term memory.** Don't be precious about ~/LocalAI/output/ — treat that as scratch.
- **The panel updates on rerun, not on a timer.** After flipping any external state (unlock, services up/down), Cmd+R the browser tab.
- **SQL is your friend.** Postgres has your whole history. docker exec -it localai-postgres psql -U localai -d localai and run SELECTs freely.
- **Save your packet ids.** create_work_packet.sh prints the UUID once. Copy it. You'll need it for answer_questions.sh.
- **Two terminals open is the minimum.** One for services + scripts, one for tail-following logs. Three is better.

21. One paragraph to remember

A vague request comes in. The system refuses to act on it. Instead it asks you five to seven questions sized to the gaps in the request. You answer them in a markdown file. The system rescores. If readiness is ≥ 0.85 (or ≥ 0.90 for high-stakes), the packet is eligible for overnight execution. Overnight, in NIGHT_MODE, the execution graph reads sources, calls local models through a safety gate, evaluates the output, retries on a secondary local model if needed, writes results to ~/LocalAI/output/<date>/, and adds memory candidates to the Obsidian review queue. Cloud is never used unless six specific conditions are explicitly met. You read the output in the morning. Nothing leaves your machine.

End of Onboarding. For the deeper architecture, see the companion System Guide PDF.